

Politechnika Krakowska im. Tadeusza Kościuszki
Wydział Informatyki i Matematyki
Informatyka, II rok, grupa GL03

Sprawozdanie z laboratorium komputerowego Metody Obliczeniowe

Ćwiczenie 11-D:
Numeryczne rozwiązanie równania dyfuzji

Autor: Jakub Nowak
Nr indeksu: 155207
Rok akademicki: 2025/2026

Spis treści

1	Wstęp i cel ćwiczenia	2
2	Opis zastosowanych metod numerycznych	2
2.1	Dyskretyzacja	2
2.2	Algorytm Thomasa	3
2.3	Metoda Gaussa-Seidela	3
3	Realizacja zadania i analiza wyników	3
3.1	Analiza rzędu dokładności (wykres 1)	3
3.2	Profile rozwiązania (wykresy 2a–2c)	4
3.3	Ewolucja błędu w czasie (wykres 3)	6
3.4	Porównanie solverów Thomasa i Gaussa-Seidela	8
4	Wnioski	8
5	Kod źródłowy	9

1 Wstęp i cel ćwiczenia

Celem ćwiczenia jest numeryczne rozwiązanie jednowymiarowego równania dyfuzji:

$$\frac{\partial U(x, t)}{\partial t} = D \frac{\partial^2 U(x, t)}{\partial x^2}, \quad x \in (-\infty, +\infty), \quad t \in [0, t_{\max}]. \quad (1)$$

Warunek początkowy to wąski pik gaussowski (2), gdzie $\tau \ll t_{\max}$ kontroluje jego szerokość; warunki brzegowe: $U(\pm\infty, t) = 0$.

$$U(x, 0) = \frac{1}{2\sqrt{\pi D \tau}} \exp\left(-\frac{x^2}{4D\tau}\right). \quad (2)$$

Dokładne rozwiązanie analityczne zachowuje kształt gaussowski:

$$U_{\text{an}}(x, t) = \frac{1}{2\sqrt{\pi D(\tau + t)}} \exp\left(-\frac{x^2}{4D(\tau + t)}\right). \quad (3)$$

Pik poszerza się i obniża z czasem — pole pod krzywą wynosi stałe 1, amplituda maleje jak $(\tau + t)^{-1/2}$. Dziedzinę nieskończoną zastąpiono przedziałem $[-a, a]$ z $a = 12 \geq 6\sqrt{D(\tau + t_{\max})} \approx 8,69$ (przy $D = 1$, $\tau = 0,1$, $t_{\max} = 2$).

Porównano trzy kombinacje metod:

1. Klasyczna Metoda Bezpośrednia (KMB) — schemat jawny, $\lambda = 0,40$;
2. Metoda Pośrednia Laasonen + algorytm Thomasa — niejawna, $\lambda = 1,00$;
3. Metoda Pośrednia Laasonen + Gauss-Seidel — niejawna, $\lambda = 1,00$.

2 Opis zastosowanych metod numerycznych

2.1 Dyskretyzacja

Siatka przestrzenna: $x_i = -a + i h$, $i = 0, \dots, N$; siatka czasowa: $t_k = k \delta t$; parametr siatki: $\lambda = D \delta t / h^2$.

KMB (schemat jawny):

$$U_i^{k+1} = \lambda U_{i-1}^k + (1 - 2\lambda) U_i^k + \lambda U_{i+1}^k. \quad (4)$$

Stabilny dla $\lambda \leq \frac{1}{2}$ (analiza von Neumanna). Przy $\lambda = 0,40$: $\delta t = 0,40 h^2$.

Laasonen:

$$-\lambda U_{i-1}^{k+1} + (1 + 2\lambda) U_i^{k+1} - \lambda U_{i+1}^{k+1} = U_i^k. \quad (5)$$

Bezwarunkowo stabilny. Przy $\lambda = 1,00$: $\delta t = 2,5 h^2$. W każdym kroku należy rozwiązać trójdzielny układ równań. Obydwa schematy mają teoretyczny rząd dokładności $\mathcal{O}(\delta t, h^2) = \mathcal{O}(h^2)$ przy stałym λ .

2.2 Algorytm Thomasa

Macierz układu (5) jest trójdzielna. Algorytm Thomasa realizuje:

1. *Eliminację w przód*: obliczenie zmodyfikowanych współczynników przekątnej i prawej strony, $\mathcal{O}(M)$;
2. *Podstawianie wstecz*: wyznaczenie rozwiązania od ostatniego węzła do pierwszego, $\mathcal{O}(M)$.

Całkowita złożoność jednego kroku: $\mathcal{O}(M)$, $M = N - 1$. Faktoryzacja wykonywana jest raz przed pętlą czasową.

2.3 Metoda Gaussa-Seidela

Jako alternatywny solver dla (5) zastosowano iteracyjną metodę Gaussa-Seidela:

$$V_j^{(m+1)} = \frac{\lambda(V_{j-1}^{(m+1)} + V_{j+1}^{(m)}) + R_j}{1 + 2\lambda}, \quad (6)$$

gdzie $R_j = U_j^k$. Kryteria stopu (spełnione jednocześnie): warunek na przyrost $|V_j^{(m+1)} - V_j^{(m)}|$ oraz na residuum układu. Progi zbieniości dobrano proporcjonalnie do h^2 , tak by dokładność solvera była porównywalna z dokładnością samego schematu.

3 Realizacja zadania i analiza wyników

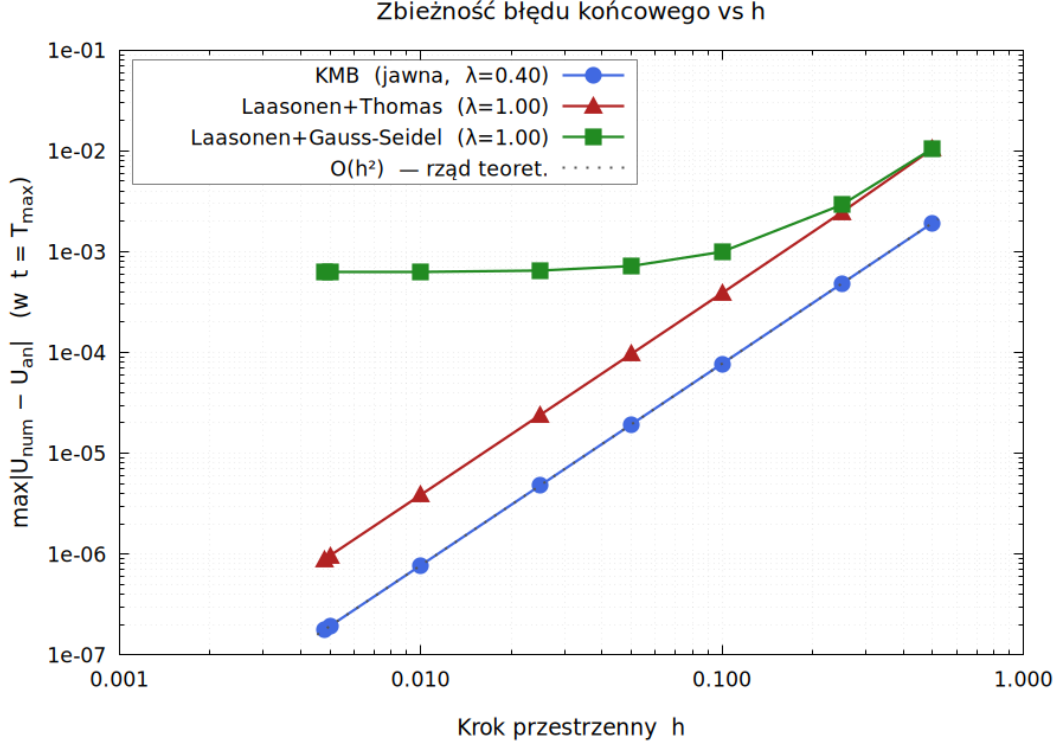
3.1 Analiza rzędu dokładności (wykres 1)

Zbadano zależność maksymalnego błędu bezwzględnego w chwili $t = t_{\max} = 2$ od kroku przestrzennego h . Empiryczny rząd zbieniości:

$$p = \frac{\ln(E_{k-1}/E_k)}{\ln(h_{k-1}/h_k)}.$$

Tabela 1: Maksymalny błąd bezwzględny $E = \max |U_{\text{num}} - U_{\text{an}}|$ w $t_{\max} = 2$ i szacowany rząd zbieniości p .

h	E_{KMB}	p_{KMB}	$E_{\text{ML+Th}}$	$p_{\text{ML+Th}}$	$E_{\text{ML+GS}}$	$p_{\text{ML+GS}}$
0.5000	$1,93 \cdot 10^{-3}$	—	$1,05 \cdot 10^{-2}$	—	$1,05 \cdot 10^{-2}$	—
0.2500	$4,83 \cdot 10^{-4}$	2.00	$2,46 \cdot 10^{-3}$	2.09	$2,92 \cdot 10^{-3}$	1.85
0.1000	$7,72 \cdot 10^{-5}$	2.00	$3,87 \cdot 10^{-4}$	2.02	$9,96 \cdot 10^{-4}$	1.17
0.0500	$1,93 \cdot 10^{-5}$	2.00	$9,66 \cdot 10^{-5}$	2.00	$7,20 \cdot 10^{-4}$	0.47
0.0250	$4,83 \cdot 10^{-6}$	2.00	$2,41 \cdot 10^{-5}$	2.00	$6,49 \cdot 10^{-4}$	0.15
0.0100	$7,72 \cdot 10^{-7}$	2.00	$3,86 \cdot 10^{-6}$	2.00	$6,29 \cdot 10^{-4}$	0.03
0.0050	$1,93 \cdot 10^{-7}$	2.00	$9,66 \cdot 10^{-7}$	2.00	$6,26 \cdot 10^{-4}$	0.01



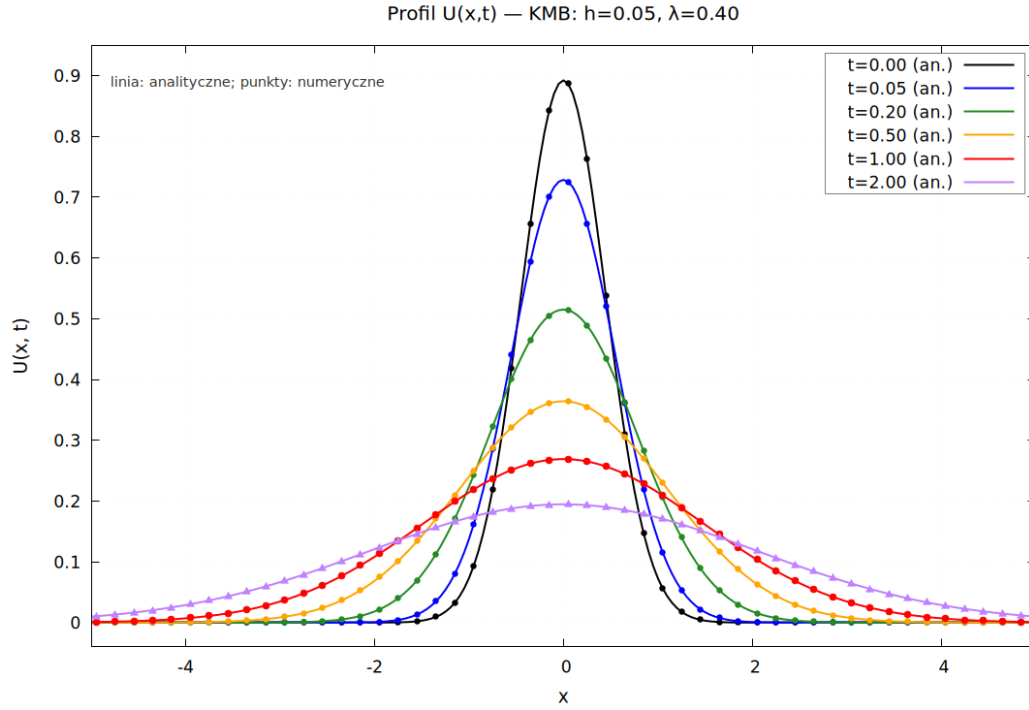
Rysunek 1: Zbieżność błędu końcowego $\max |U_{\text{num}} - U_{\text{an}}|$ w $t = t_{\text{max}}$ w funkcji kroku h (skala log-log). Linia przerywana: teoretyczny rząd $\mathcal{O}(h^2)$.

Z tabeli i wykresu (rys. 1) wyraźnie widać, że:

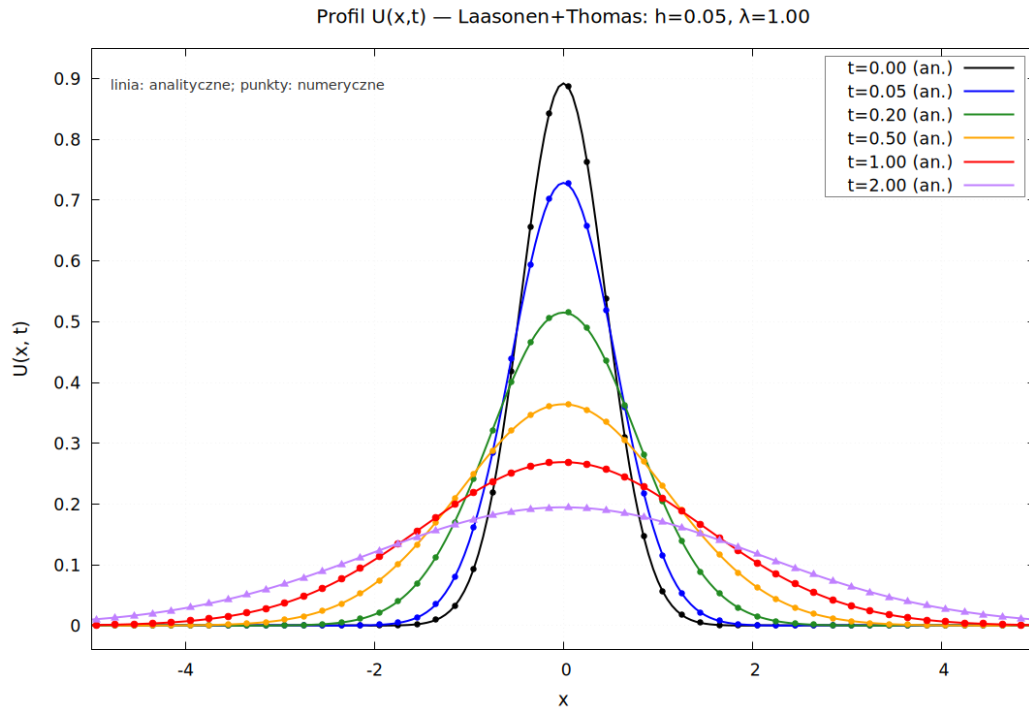
- **KMB** osiąga dokładnie rząd $p = 2,0000$ dla wszystkich wartości h — krzywa na wykresie log-log przebiega idealnie równolegle do linii referencyjnej $\mathcal{O}(h^2)$. Pełna zgodność z teorią dla schematu jawnego drugiego rzędu.
- **Laasonen + Thomas** również zachowuje $p \approx 2,00$. Metoda ta jest mniej dokładna niż KMB przy tym samym h — wynika to z większego λ : $\delta t_{\text{ML}} = 2,5 h^2 > \delta t_{\text{KMB}} = 0,4 h^2$, co przekłada się na większy błąd czasowy schematu.
- **Laasonen + Gauss-Seidel** wykazuje wyraźnie odchylenie od $\mathcal{O}(h^2)$: dla $h \leq 0,05$ błąd przestaje maleć i stabilizuje się na poziomie $\approx 6,26 \times 10^{-4}$. Jest to konsekwencja natury metod iteracyjnych — GS zatrzymuje się po spełnieniu kryterium stopu, nie rozwiązując układu dokładnie. W każdym kroku czasowym pojawia się więc mały błąd solvera, który wraz z kolejnymi krokami narasta i zaczyna dominować nad błędem schematu — zagęszczanie siatki nie przynosi wtedy poprawy.

3.2 Profile rozwiązania (wykresy 2a–2c)

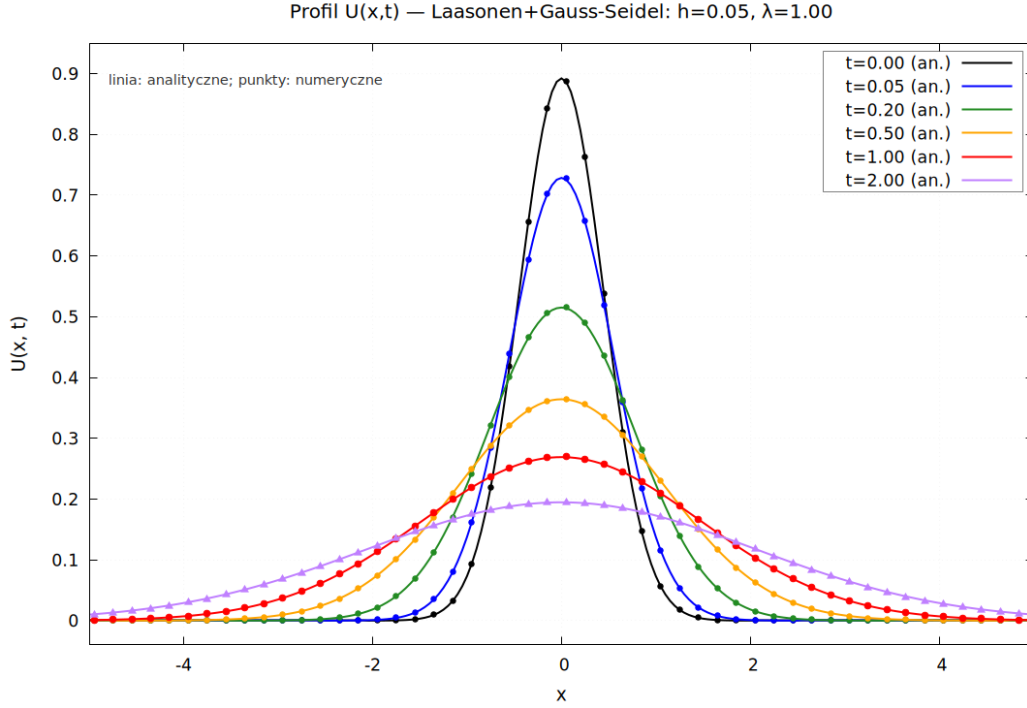
Na rysunkach 2–4 pokazano profile $U(x, t_i)$ dla $t \in \{0,00; 0,05; 0,20; 0,50; 1,00; 2,00\}$ przy $h = 0,05$. Punkty: rozwiązanie numeryczne (co czwarty węzeł); linia ciągła: rozwiązanie analityczne.



Rysunek 2: Profile $U(x,t)$ — KMB ($h = 0,05$, $\lambda = 0,40$).



Rysunek 3: Profile $U(x,t)$ — Laasonen + Thomas ($h = 0,05$, $\lambda = 1,00$).

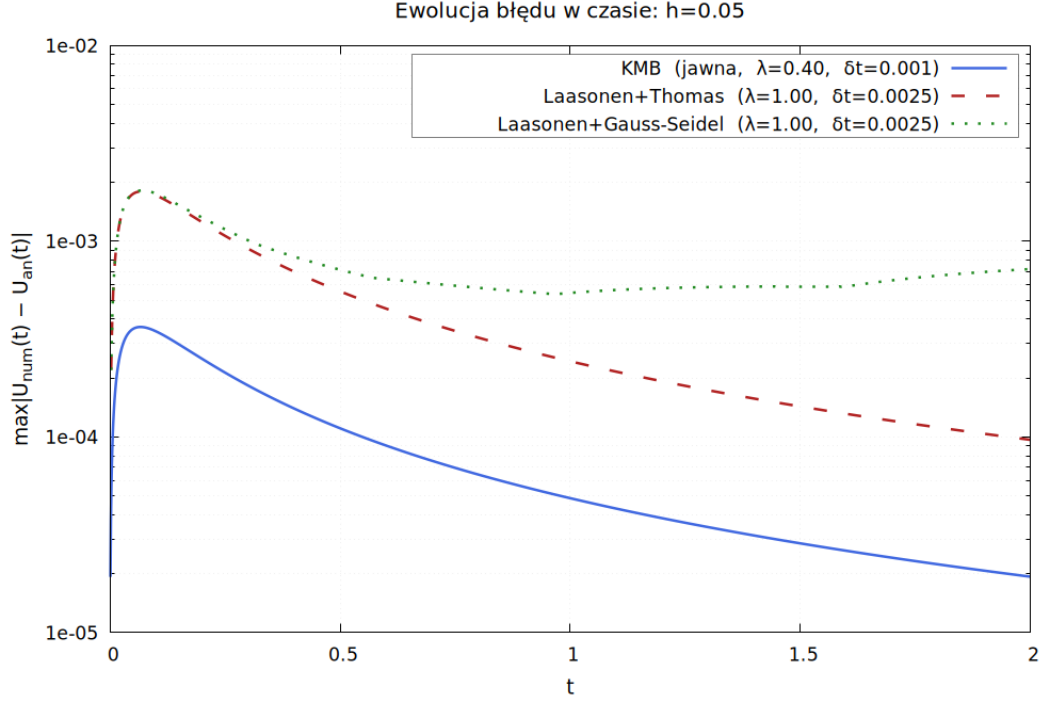


Rysunek 4: Profile $U(x, t)$ — Laasonen + Gauss-Seidel ($h = 0,05$, $\lambda = 1,00$).

KMB (rys. 2) i Laasonen+Thomas (rys. 3) pokazują bardzo dobrą zgodność z analizą: punkty pokrywają się z liniami we wszystkich chwilach. Gaussowski pik przy $t = 0$ stopniowo poszerza się i obniża, zgodnie ze wzorem (3). Dla Laasonen + GS (rys. 4) profile wyglądają wizualnie identycznie jak dla pozostałych metod — różnica błędów jest widoczna dopiero w tabeli i na wykresie błędu, nie na profilach.

3.3 Ewolucja błędu w czasie (wykres 3)

Rysunek 5 pokazuje $\max_i |U_i^k - U_{\text{an}}(x_i, t_k)|$ w funkcji czasu t przy $h = 0,05$.



Rysunek 5: Ewolucja maksymalnego błędu bezwzględnego w czasie ($h = 0,05$, oś y w skali log).

Zachowanie KMB i Laasonen+Thomas.

Obie krzywe wykazują niezerowy wzrost na początku, osiągają maksimum, a następnie monotonicznie maleją. Wyjaśnienie:

1. *Wzrost na początku* — w chwili $t = 0$ gaussowski pik jest bardzo wąski (szerokość $\sim \sqrt{D\tau} \approx 0,32$), więc rozwiązanie zmienia się gwałtownie w przestrzeni. Im ostrzejszy kształt, tym większy błąd dyskretyzacji na danej siatce. Błąd rośnie od zera, gdyż warunek początkowy jest nałożony bez błędu.
2. *Maksimum* — dla KMB szczyt błędu pojawia się w okolicach $t^* \approx 0,066 \approx \frac{2}{3}\tau$, gdzie wynosi $\approx 3,64 \times 10^{-4}$. Dla ML+Thomas szczyt wypada nieco później ze względu na większy krok czasowy ($\delta t_{ML} = 2,5 \cdot (0,05)^2 = 6,25 \times 10^{-3}$).
3. *Malejący ogon* — z upływem czasu pik rozszerza się i wygadza, więc siatka coraz lepiej go odwzorowuje i błąd systematycznie maleje.

Zachowanie Laasonen + GS.

Krzywa GS od pierwszych kroków jest znacznie wyżej niż KMB i ML+Thomas i pozostaje na podwyższonym, w przybliżeniu stałym poziomie przez cały przedział $[0, t_{\max}]$. To potwierdzenie obserwacji z sekcji 3.1: skumulowane błędy solvera dominują nad błędem schematu.

3.4 Porównanie solverów Thomasa i Gaussa-Seidela

Tabela 2: Porównanie solverów dla metody Laasonen, $h = 0,05$.

Solver	$\max E (t_{\max})$	Czas T [s]
Algorytm Thomasa	$9,66 \times 10^{-5}$	0.004
Gauss-Seidel	$7,20 \times 10^{-4}$	0.037

Thomas jest prawie $10\times$ dokładniejszy i $\sim 9\times$ szybszy przy $h = 0,05$. Dla mniejszych h dysproporcja rośnie — Gauss-Seidel wykonuje wiele iteracji na krok, podczas gdy Thomas ma stałe $\mathcal{O}(M)$.

4 Wnioski

1. **Poprawność implementacji.** KMB i Laasonen+Thomas osiągają empiryczny rząd zbieniości $p \approx 2,00$, co jest w pełni zgodne z teorią.
2. **Porównanie metod dyskretyzacji.** KMB jest prostsza w implementacji i przy tym samym h używa mniejszego kroku czasowego δt , co przekłada się na mniejszy błąd, lecz wymaga spełnienia warunku stabilności $\lambda \leq 0,5$. Laasonen jest bezwarunkowo stabilna i pozwala stosować większy δt kosztem nieco wyższego błędu całkowitego.
3. **Kluczowa rola solvera.** Algorytm Thomasa jest optymalny dla macierzy trójdzielnych — złożoność $\mathcal{O}(M)$ i błąd na poziomie precyzji maszynowej. Solver GS z tolerancjami $\sim h^2$ nie zapewnia wystarczającej dokładności dla małych h : skumulowany błąd solvera ustala się na stałym poziomie i ogranicza efektywną zbieniość schematu.
4. **Dynamika błędu w czasie.** Błąd osiąga maksimum około $t^* \approx \frac{2}{3}\tau \approx 0,067$, po czym monotonicznie maleje. Wynika to z wąskiego piku gaussowskiego w $t = 0$, który jest trudny do odwzorowania na skończonej siatce; wraz z upływem czasu pik się rozszerza i wygadza, zmniejszając błąd dyskretyzacji.
5. **Wizualna walidacja.** Profile numeryczne KMB i Laasonen+Thomas pokrywają się z rozwiązaniem analitycznym dla wszystkich sześciu badanych chwil, potwierdzając poprawność implementacji.

5 Kod źródłowy

Poniżej zamieszczono pełny kod źródłowy programu main.cpp, skompilowanego poleceniem:

```
g++ -std=c++17 -O2 -o lab11d main.cpp -lm
```

```
1  #include <cstdio>
   #include <cmath>
   #include <vector>
   #include <ctime>
   using namespace std;

6
   /* PARAMETRY FIZYCZNE */
   const double D      = 1.0;
   const double TAU     = 0.1;
   const double TMAX    = 2.0;
11  const double A_DOM = 12.0;      /* a >= 6*sqrt(D*(TAU+TMAX)) */

   /* WSPOLCZYNNIKI SCHEMATOW */
   const double LAM_KMB = 0.40;    /* stabilnosc KMB: lambda <= 0.5 */
   const double LAM_ML  = 1.00;    /* Laasonen: bezwarunkowo stabilny */

16
   /* KRYTERIA STOPU GAUSSA-SEIDELA */
   const double TOL_GS_DX_BAZA = 1.0e-12;
   const double TOL_GS_R_BAZA  = 1.0e-11;
   const int    MAX_GS_IT      = 5000;

21
   /* ROZWIAZANIE ANALITYCZNE */
   double U_dokladne(double x, double t)
   {
       return exp(-x * x / (4.0 * D * (TAU + t)))
              / (2.0 * sqrt(M_PI * D * (TAU + t)));
26   }

   /* PARAMETRY SIATKI */
   struct Siatka
31   {
       int    N, M, Nt;
       double h, dt, lam;
   };

36   Siatka utworz_siatke(double h_zad, double lam_zad)
   {
       Siatka s;

       s.N = (int) round(2.0 * A_DOM / h_zad);
41       if(s.N < 4) s.N = 4;

       s.h  = 2.0 * A_DOM / s.N;
       s.dt = lam_zad * s.h * s.h / D;
       s.Nt = (int) ceil(TMAX / s.dt);
46       if(s.Nt < 1) s.Nt = 1;

       s.dt  = TMAX / s.Nt;
       s.lam = D * s.dt / (s.h * s.h);
       s.M   = s.N - 1;

51   }
```

```

        return s;
    }

    /* SIATKA PRZESTRZENNA */
56 void siatka_x(const Siatka& s, vector<double>& x)
    {
        x.resize(s.N + 1);
        for(int i = 0; i <= s.N; i++) x[i] = -A_DOM + i * s.h;
    }

61 /* WARUNEK POEZATKOWY */
void warunki_poczatkowe(const Siatka& s, const vector<double>& x,
    vector<double>& U)
    {
        U.assign(s.N + 1, 0.0);
66     for(int i = 0; i <= s.N; i++) U[i] = U_dokladne(x[i], 0.0);
        U[0] = U[s.N] = 0.0;
    }

    /* MAKSYMALNY BLAD BEZWZGLEDNY */
71 double blad_maks(const Siatka& s, const vector<double>& x, const
    vector<double>& U, double t)
    {
        double blad = 0.0;

        for(int i = 1; i < s.N; i++)
76     {
            double e = fabs(U[i] - U_dokladne(x[i], t));
            if(e > blad) blad = e;
        }

81     return blad;
    }

    /* ZAPIS PROFILU U(x) */
void zapisz_profil(FILE *f, const Siatka& s, const vector<double>& x
    , const vector<double>& U, double t)
86 {
    for(int i = 1; i < s.N; i++)
        fprintf(f, "%.15e\t%.15e\t%.15e\n", x[i], U[i], U_dokladne(x
[i], t));

    fprintf(f, "\n\n");
91 }

    /* ALGORYTM THOMASA -- FAKTORYZACJA */
void thomas_fak(const vector<double>& dol, const vector<double>& gl,
    const vector<double>& gor,
        vector<double>& gl_fak, vector<double>& mu)
96 {
    int M = (int) gl.size();

    gl_fak = gl;
    for(int j = 1; j < M; j++)
101 {
        mu[j] = dol[j] / gl_fak[j - 1];
        gl_fak[j] -= mu[j] * gor[j - 1];
    }
}

```

```

}
106
/* ALGORYTM THOMASA -- ROZWIAZANIE */
void thomas_rozw(const vector<double>& gor, const vector<double>&
    gl_fak, const vector<double>& mu,
    vector<double>& praw, vector<double>& y)
{
111     int M = (int) gl_fak.size();

    for(int j = 1; j < M; j++)
        praw[j] -= mu[j] * praw[j - 1];

116     y[M - 1] = praw[M - 1] / gl_fak[M - 1];
    for(int j = M - 2; j >= 0; j--)
        y[j] = (praw[j] - gor[j] * y[j + 1]) / gl_fak[j];
}

121 // Metoda Gaussa-Seidela dla ukladu Laasonen
int gauss_seidel(const vector<double>& R, double lam, vector<double>
    & V, double h)
{
    int M = (int) V.size();
    double den = 1.0 + 2.0 * lam;

126     vector<double> V_poprz(M);
    vector<double> tol_dx(M), tol_r(M);

    double tol_dx_b = fmax(TOL_GS_DX_BAZA, 1.0e-3 * h * h);
131     double tol_r_b = fmax(TOL_GS_R_BAZA, 1.0e-3 * h * h);

    for(int j = 0; j < M; j++)
    {
136         tol_dx[j] = tol_dx_b;
        tol_r[j] = tol_r_b;
    }

    for(int it = 0; it < MAX_GS_IT; it++)
    {
141         for(int j = 0; j < M; j++) V_poprz[j] = V[j];

        for(int j = 0; j < M; j++)
        {
146             double vl = (j > 0) ? V[j - 1] : 0.0;
            double vr = (j < M - 1) ? V[j + 1] : 0.0;
            V[j] = (lam * (vl + vr) + R[j]) / den;
        }

        bool wszystkie_tolerancje = true;
151         for(int j = 0; j < M; j++)
        {
            double vl = (j > 0) ? V[j - 1] : 0.0;
            double vr = (j < M - 1) ? V[j + 1] : 0.0;
            double residuum = R[j] - (den * V[j] - lam * vl - lam *
vr);

156             wszystkie_tolerancje =
                wszystkie_tolerancje &&
                (fabs(V[j] - V_poprz[j]) < tol_dx[j]) &&

```

```

161         (fabs(residuum) < tol_r[j]));
    }

    if(wszystkie_tolerancje) return it + 1;
}

166 return -1;
}

/* KMB -- METODA BEZPOSREDNIA */
double rozwarz_KMB(double h_zad, FILE *f_hist, FILE *f_snap,
171 const vector<double> *snap_t, double *czas_s)
{
    Siatka s = utworz_siatke(h_zad, LAM_KMB);

    vector<double> x;
    vector<double> U;
176 vector<double> Un(s.N + 1, 0.0);

    siatka_x(s, x);
    warunki_początkowe(s, x, U);

181 int si = 0;
    if(f_snap && snap_t && !snap_t->empty() && (*snap_t)[0] <= 1.0e
-12)
    {
        zapisz_profil(f_snap, s, x, U, 0.0);
186 si = 1;
    }

    if(f_hist) fprintf(f_hist, "0.0\t0.0\n");

191 clock_t t0 = clock();
    for(int k = 0; k < s.Nt; k++)
    {
        double t = (k + 1) * s.dt;

196 for(int i = 1; i < s.N; i++)
        Un[i] = s.lam * U[i - 1] + (1.0 - 2.0 * s.lam) * U[i] +
s.lam * U[i + 1];

        Un[0] = Un[s.N] = 0.0;
        U.swap(Un);

201 if(f_hist)
        fprintf(f_hist, "%.15e\t%.15e\n", t, blad_maks(s, x, U,
t));

        if(f_snap && snap_t && si < (int) snap_t->size() && t >= (*
snap_t)[si] - 0.5 * s.dt)
206 {
            zapisz_profil(f_snap, s, x, U, t);
            si++;
        }
    }

211 clock_t t1 = clock();
    if(czas_s) *czas_s = (double) (t1 - t0) / CLOCKS_PER_SEC;

```

```

216     return blad_maks(s, x, U, TMAX);
}

/* LAASONEN + ALGORYTM THOMASA */
double rozwiaz_ML_thomas(double h_zad, FILE *f_hist, FILE *f_snap,
                        const vector<double> *snap_t, double *
221     czas_s)
{
    Siatka s = utworz_siatke(h_zad, LAM_ML);

    vector<double> x;
    vector<double> U;

226     siatka_x(s, x);
    warunki_poczatkowe(s, x, U);

    vector<double> dol(s.M), gl(s.M), gor(s.M);
231     for(int j = 0; j < s.M; j++)
    {
        dol[j] = (j > 0) ? s.lam : 0.0;
        gl[j] = -(1.0 + 2.0 * s.lam);
        gor[j] = (j < s.M - 1) ? s.lam : 0.0;
236     }

    vector<double> gl_fak(s.M), mu(s.M);
    thomas_fak(dol, gl, gor, gl_fak, mu);

241     vector<double> praw(s.M), V(s.M);

    int si = 0;
    if(f_snap && snap_t && !snap_t->empty() && (*snap_t)[0] <= 1.0e
-12)
    {
246         zapisz_profil(f_snap, s, x, U, 0.0);
        si = 1;
    }

    if(f_hist) fprintf(f_hist, "0.0\t0.0\n");

251     clock_t t0 = clock();
    for(int k = 0; k < s.Nt; k++)
    {
        double t = (k + 1) * s.dt;

256         for(int j = 0; j < s.M; j++) praw[j] = -U[j + 1];
        thomas_rozw(gor, gl_fak, mu, praw, V);

        for(int j = 0; j < s.M; j++) U[j + 1] = V[j];
261         U[0] = U[s.N] = 0.0;

        if(f_hist)
            fprintf(f_hist, "%.15e\t%.15e\n", t, blad_maks(s, x, U,
t));

266         if(f_snap && snap_t && si < (int) snap_t->size() && t >= (*
snap_t)[si] - 0.5 * s.dt)
        {

```

```

        zapisz_profil(f_snap, s, x, U, t);
        si++;
    }
271 }

    clock_t t1 = clock();
    if(czas_s) *czas_s = (double) (t1 - t0) / CLOCKS_PER_SEC;

276    return blad_maks(s, x, U, TMAX);
}

/* LAASONEN + GAUSS-SEIDEL */
double rozwarz_ML_gs(double h_zad, FILE *f_hist, FILE *f_snap,
281     const vector<double> *snap_t, double *czas_s)
{
    Siatka s = utworz_siatke(h_zad, LAM_ML);

    vector<double> x;
286    vector<double> U;
    vector<double> praw(s.M), V(s.M, 0.0);

    siatka_x(s, x);
    warunki_początkowe(s, x, U);

291    int si = 0;
    if(f_snap && snap_t && !snap_t->empty() && (*snap_t)[0] <= 1.0e
-12)
    {
        zapisz_profil(f_snap, s, x, U, 0.0);
296        si = 1;
    }

    if(f_hist) fprintf(f_hist, "0.0\t0.0\n");

301    clock_t t0 = clock();
    for(int k = 0; k < s.Nt; k++)
    {
        double t = (k + 1) * s.dt;

306        for(int j = 0; j < s.M; j++) praw[j] = U[j + 1];

        int iter_gs = gauss_seidel(praw, s.lam, V, s.h);
        if(iter_gs < 0)
            fprintf(stderr, "OSTRZEZENIE: GS nie spelnil wszystkich
tolerancji w kroku czasu %d\n", k + 1);

311        for(int j = 0; j < s.M; j++) U[j + 1] = V[j];
        U[0] = U[s.N] = 0.0;

        if(f_hist)
316            fprintf(f_hist, "%.15e\t%.15e\n", t, blad_maks(s, x, U,
t));

        if(f_snap && snap_t && si < (int) snap_t->size() && t >= (*
snap_t)[si] - 0.5 * s.dt)
        {
            zapisz_profil(f_snap, s, x, U, t);
321            si++;
        }
    }
}

```

```

    }
}

clock_t t1 = clock();
326 if(czas_s) *czas_s = (double) (t1 - t0) / CLOCKS_PER_SEC;

return blad_maks(s, x, U, TMAX);
}

331 int main()
{
    FILE *plik;

    const vector<double> snap_t = {0.0, 0.05, 0.2, 0.5, 1.0, 2.0};
336 const double h_vals[] = {0.50, 0.25, 0.10, 0.05, 0.025, 0.010,
0.005, 0.0048};
    const int n_h = (int) (sizeof(h_vals) / sizeof(h_vals[0]));

    printf("Cwiczenie 11-D: rownanie dyfuzji 1D\n");
    printf("D = %.1f, tau = %.1f, Tmax = %.1f, A = %.1f\n", D, TAU,
TMAX, A_DOM);
341 printf("KMB: lambda = %.2f      ML: lambda = %.1f\n\n", LAM_KMB,
LAM_ML);

    /* (1) ZBIEZNOSC: max/blad/(Tmax) vs h */
    {
        double e_kmb_prev = 0.0;
346 double e_ml_prev = 0.0;
        double e_gs_prev = 0.0;
        double h_prev = 0.0;

        plik = fopen("konwergencja.dat", "w");
351 fprintf(plik, "# h   blad_KMB   t_KMB[s]   blad_ML   t_ML[s]
blad_ML_GS   t_ML_GS[s]\n");

        printf("%8s %16s %16s %16s %10s %10s %10s\n",
               "h", "blad_KMB", "blad_ML", "blad_ML_GS", "p_KMB", "
p_ML", "p_GS");
        printf("
-----
n");
356

        for(int k = 0; k < n_h; k++)
        {
            double h = h_vals[k];
            double t_kmb, t_ml, t_gs;

361 double e_kmb = rozwarz_KMB(h, NULL, NULL, NULL, &t_kmb);
            double e_ml = rozwarz_ML_thomas(h, NULL, NULL, NULL, &
t_ml);
            double e_gs = rozwarz_ML_gs(h, NULL, NULL, NULL, &t_gs)
;

366 double p_kmb = (e_kmb_prev > 0.0) ? log(e_kmb_prev /
e_kmb) / log(h_prev / h) : 0.0;
            double p_ml = (e_ml_prev > 0.0) ? log(e_ml_prev /
e_ml ) / log(h_prev / h) : 0.0;
            double p_gs = (e_gs_prev > 0.0) ? log(e_gs_prev /

```



```

e_gs ) / log(h_prev / h) : 0.0;

    fprintf(plik, "%.15e %.15e %.15e %.15e %.15e %.15e
%.15e\n",
371         h, e_kmb, t_kmb, e_ml, t_ml, e_gs, t_gs);

    if(k == 0) {
        printf("%8.4f %16.6e %16.6e %16.6e %10s %10s %10s\n"
,
        h, e_kmb, e_ml, e_gs, "---", "---", "---");
376    } else {
        printf("%8.4f %16.6e %16.6e %16.6e %10.4f %10.4f
%10.4f\n",
        h, e_kmb, e_ml, e_gs, p_kmb, p_ml, p_gs);
    }

    e_kmb_prev = e_kmb;
    e_ml_prev  = e_ml;
    e_gs_prev  = e_gs;
    h_prev     = h;
}

386
fclose(plik);
printf("\nZapisano: konwergencja.dat\n");
printf("UWAGA: p ~ 2 oznacza zgodnosc z teoretycznym rzadem
0(h^2).\n\n");
}

391
/* (2) PROFILE U(x,t_i) */
{
    double h = 0.05;

396
    plik = fopen("profil_kmb.dat", "w");
    fprintf(plik, "# KMB h = %.2f, lam = %.2f\n# x  U_num  U_an\
n", h, LAM_KMB);
    rozwarz_KMB(h, NULL, plik, &snap_t, NULL);
    fclose(plik);
    printf("Zapisano: profil_kmb.dat (h = %.2f)\n", h);

401
    plik = fopen("profil_ml.dat", "w");
    fprintf(plik, "# ML+Thomas h = %.2f, lam = %.2f\n# x  U_num
U_an\n", h, LAM_ML);
    rozwarz_ML_thomas(h, NULL, plik, &snap_t, NULL);
    fclose(plik);
406
    printf("Zapisano: profil_ml.dat (h = %.2f)\n", h);

    plik = fopen("profil_ml_gs.dat", "w");
    fprintf(plik, "# ML+Gauss-Seidel h = %.2f, lam = %.2f\n# x
U_num  U_an\n", h, LAM_ML);
    rozwarz_ML_gs(h, NULL, plik, &snap_t, NULL);
    fclose(plik);
411
    printf("Zapisano: profil_ml_gs.dat (h = %.2f)\n\n", h);
}

/* (3) EWOLUCJA BLEDU W CZASIE */
416
{
    double h = 0.05;

```

```

plik = fopen("blad_t_kmb.dat", "w");
fprintf(plik, "# t    max_blad    (KMB, h = %.2f, lam = %.2f)\n"
, h, LAM_KMB);
421   rozwarz_KMB(h, plik, NULL, NULL, NULL);
      fclose(plik);
      printf("Zapisano: blad_t_kmb.dat (h = %.2f)\n", h);

      plik = fopen("blad_t_ml.dat", "w");
426   fprintf(plik, "# t    max_blad    (ML+Thomas, h = %.2f, lam =
      %.2f)\n", h, LAM_ML);
      rozwarz_ML_thomas(h, plik, NULL, NULL, NULL);
      fclose(plik);
      printf("Zapisano: blad_t_ml.dat      (h = %.2f)\n", h);

      plik = fopen("blad_t_ml_gs.dat", "w");
431   fprintf(plik, "# t    max_blad    (ML+Gauss-Seidel, h = %.2f,
      lam = %.2f)\n", h, LAM_ML);
      rozwarz_ML_gs(h, plik, NULL, NULL, NULL);
      fclose(plik);
      printf("Zapisano: blad_t_ml_gs.dat (h = %.2f)\n\n", h);
436   }

      /* POROWNANIE: THOMAS VS GAUSS-SEIDEL */
      {
          double h = 0.05;
          double t_thomas, t_gs;

          double e_thomas = rozwarz_ML_thomas(h, NULL, NULL, NULL, &
t_thomas);
          double e_gs      = rozwarz_ML_gs(h, NULL, NULL, NULL, &t_gs);

          printf("Porownanie ML (h = %.2f):\n", h);
          printf("  Thomas:      blad = %.6e,  czas = %.3f s\n",
e_thomas, t_thomas);
          printf("  Gauss-Seidel: blad = %.6e,  czas = %.3f s\n", e_gs
, t_gs);
          printf("  roznica bledow: %.2e\n\n", fabs(e_thomas - e_gs));
          }

451   printf("Generowanie wykresow: gnuplot plot1.gp plot2.gp plot3.gp
      \n");
      return 0;
}

```

Listing 1: main.cpp — pełny kod źródłowy programu